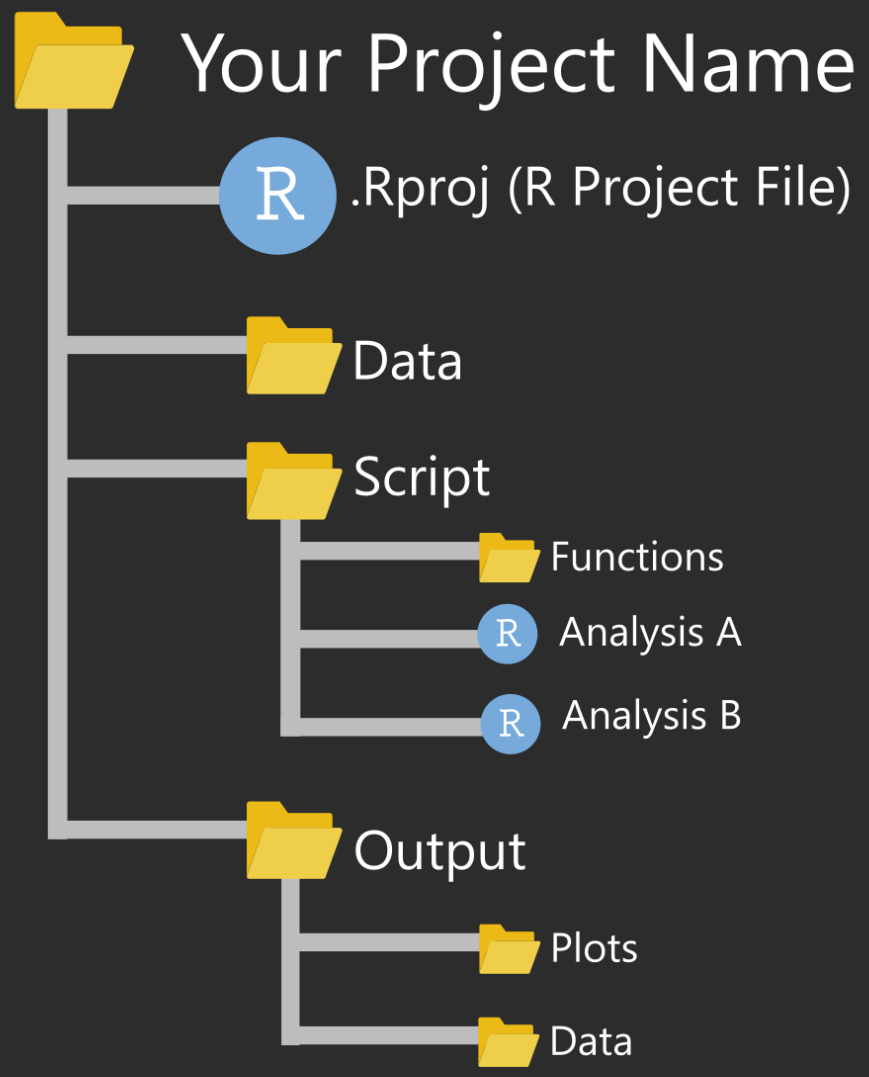
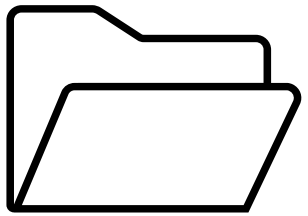
Kako narediti R analize čitljive, razumljive in ponovljive

Organizacija dela v R Studiu

A basic R project set up



Struktura - enostaven projekt

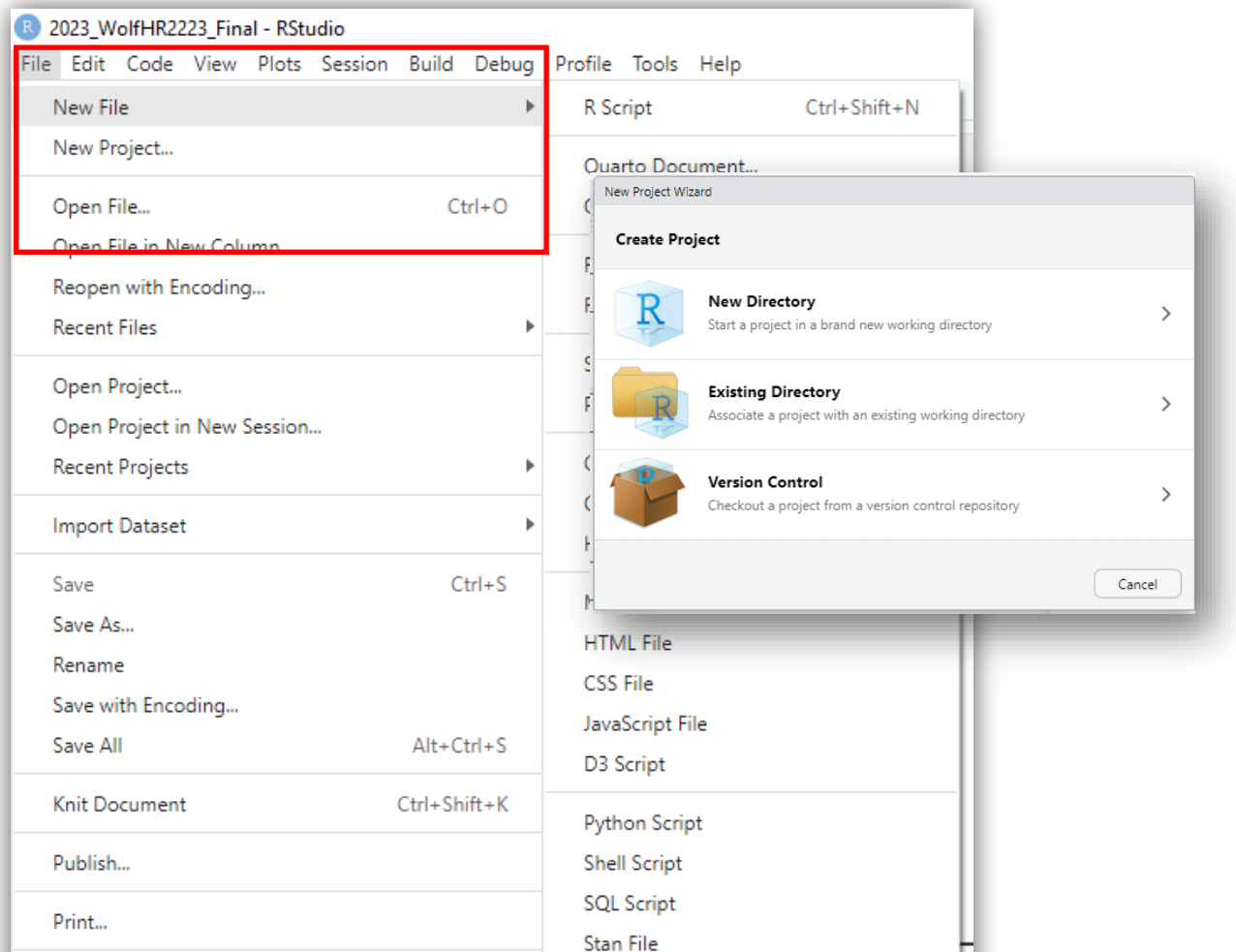


MojProjekt

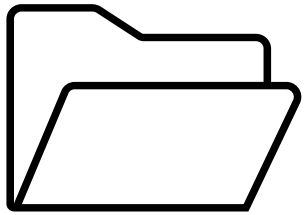
MojProjekt.Rproj

MojiPodatki.txt

Rstudio – File – Create Project



Struktura - enostaven projekt



MojProjekt

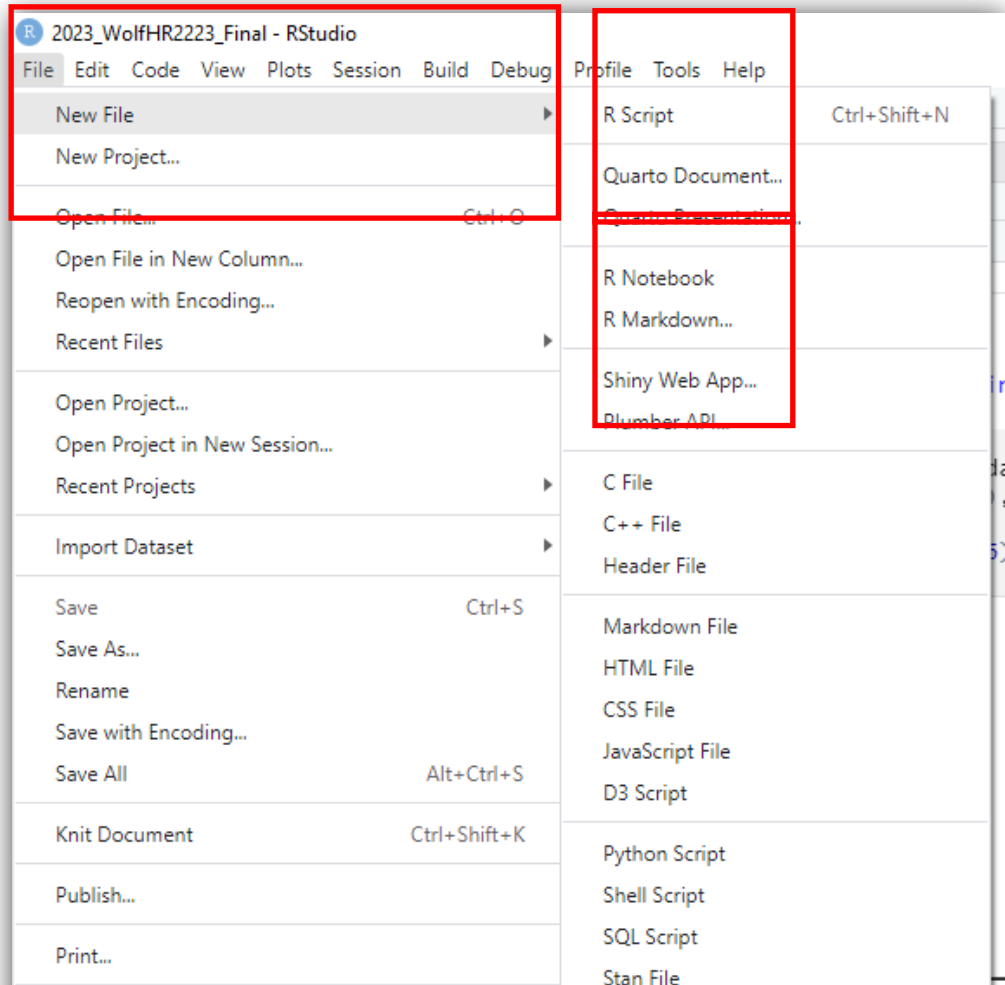
MojProjekt.Rproj

MojiPodatki.txt

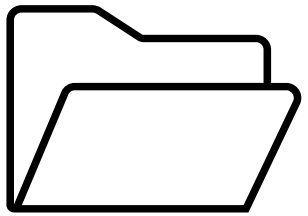
MojaAnaliza.Rmd

MojaSkripta.R

Rstudio – File – New File



Struktura - enostaven projekt



MojProjekt

MojProjekt.Rproj

MojiPodatki.txt

MojaAnaliza.Rmd

MojaSkripta.R

Če so podatki in v isti mapi....

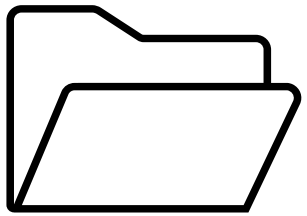
Enostavno nalaganje podatkov v R:

```
read.table("MojiPodatki.txt")
```

Enostavno pisanje rezultatov na disk:

```
write.xlsx("TabelaRezultatov.xlsx")
```

Struktura - enostaven projekt



MojProjekt

MojProjekt.Rproj

MojiPodatki.txt

MojaAnaliza.Rmd

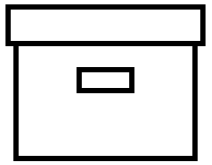
MojaSkripta.R

MojaAnaliza.pdf

TabelaRezultatov.xlsx

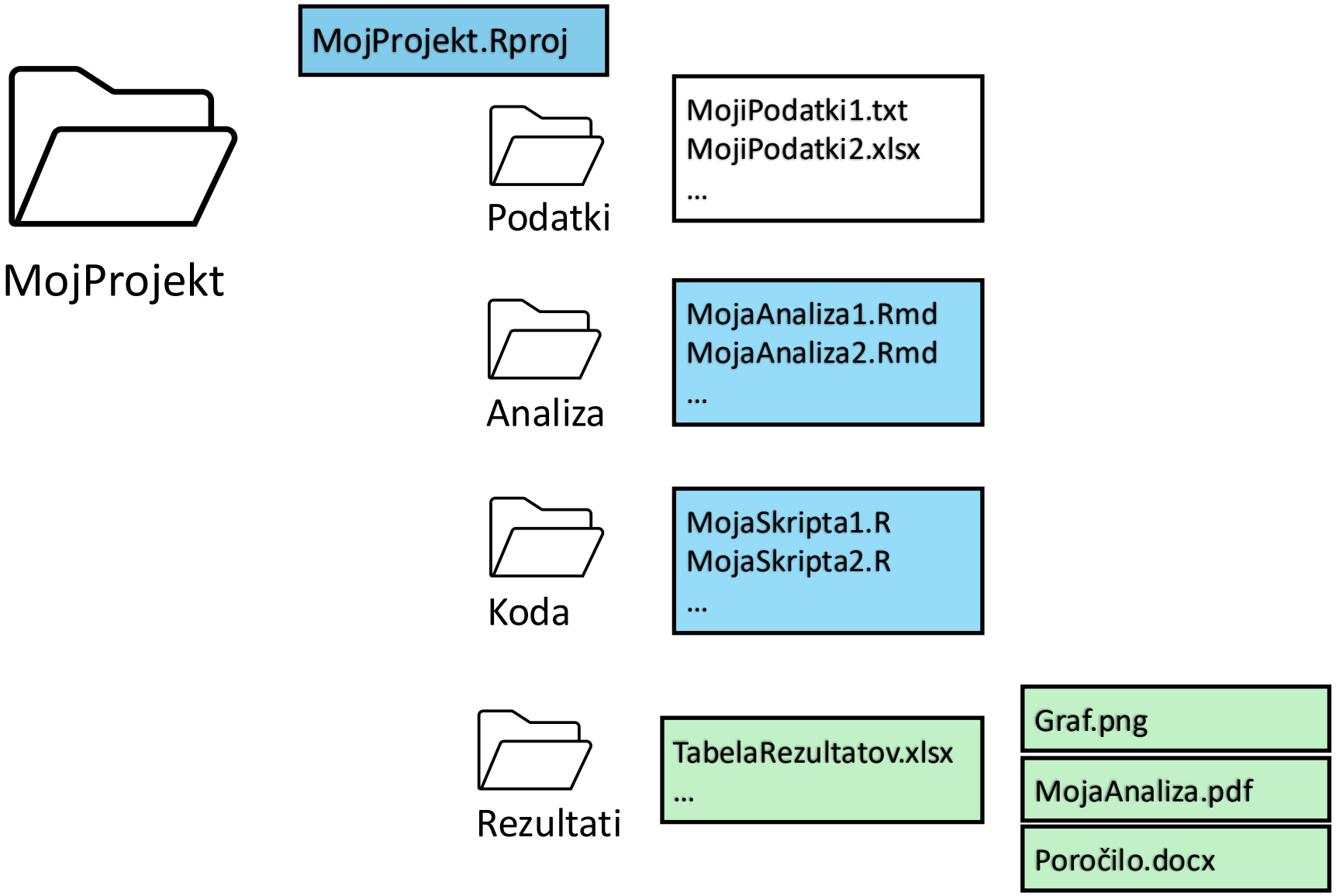
Graf.png

Vse urejeno – lahko se naredi .zip in pošlje

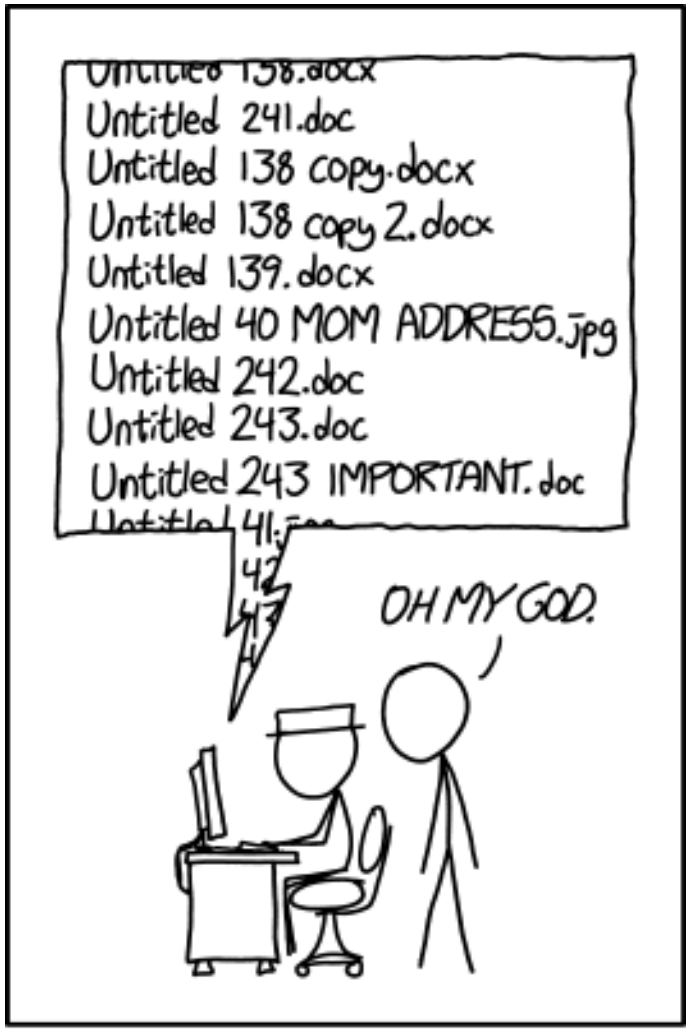


MojProjekt.zip

Struktura – bolj kompleksen projekt



Imena datotek
(in možnost neskončnih bolečin)



PROTIP: NEVER LOOK IN SOMEONE ELSE'S DOCUMENTS FOLDER.

Uporabljajte **informativna imena** za projekte, mape in datoteke!

WolfParentage2024

WolfParentageDataAnalsis2024.Rproj

Data

WolfGenotypes2024.txt
SampleMetadata_Wolf2024.xlsx

Analysis

01_DataPreparation.Rmd
02_ParentageAnalysis.Rmd
03_Capture_Mark_Recapture.RMD

Code

PedigreeFunctions.R
PrepareColonyData.R
PopulationDynamicsFunctions.R

Results

GeneticDiversity_Wolf2024.xlsx
PedigreeGraph_Wolf2024.png
MarkRecaptureResults_Wolf2024.xlsx

Language
Linguaggio
ЯЗЫК
Язык
Γλώσσα
Język
بولى
لسان
Lenguaje
Langage
Jezik?
言語
भाषा
Ngôn ngữ
اللغة
Linguagem
Wika
Sprache
语言
Bahasa
언어

Uporabljajte **informativna imena** za projekte, mape in datoteke!

WolfParentage2024

WolfParentageDataAnalsis2024.Rproj

Data

WolfGenotypes2024.txt
SampleMetadata_Wolf2024.xlsx

Analysis

01_DataPreparation.Rmd
02_ParentageAnalysis.Rmd
03_Capture_Mark_Recapture.RMD

Code

PedigreeFunctions.R
PrepareColonyData.R
PopulationDynamicsFunctions.R

Results

GeneticDiversity_Wolf2024.xlsx
PedigreeGraph_Wolf2024.png
MarkRecaptureResults_Wolf2024.xlsx

Slabo!!!

Projekt4

Data1.txt
RG_lvfmx.xlsx
Analysis1.Rmd
Analsis2.Rmd
Analysis2.copy.Rmd
Functions.R
Functions_Backup.R
Results2.xlsx

Picture1.jpg
Picture2.jpg
Projekt4.Rproj
FilteredData22.csv
Graph1.pdf
Table.xls
Output3.tmp.xlsx

Organizacija dela - povzetek

Cilji:

Prenosljivost: Celotno analizo in podatke lahko v eni potezi “zazipate” in pošljete, pa bodo še vedno vse skripte delovale.

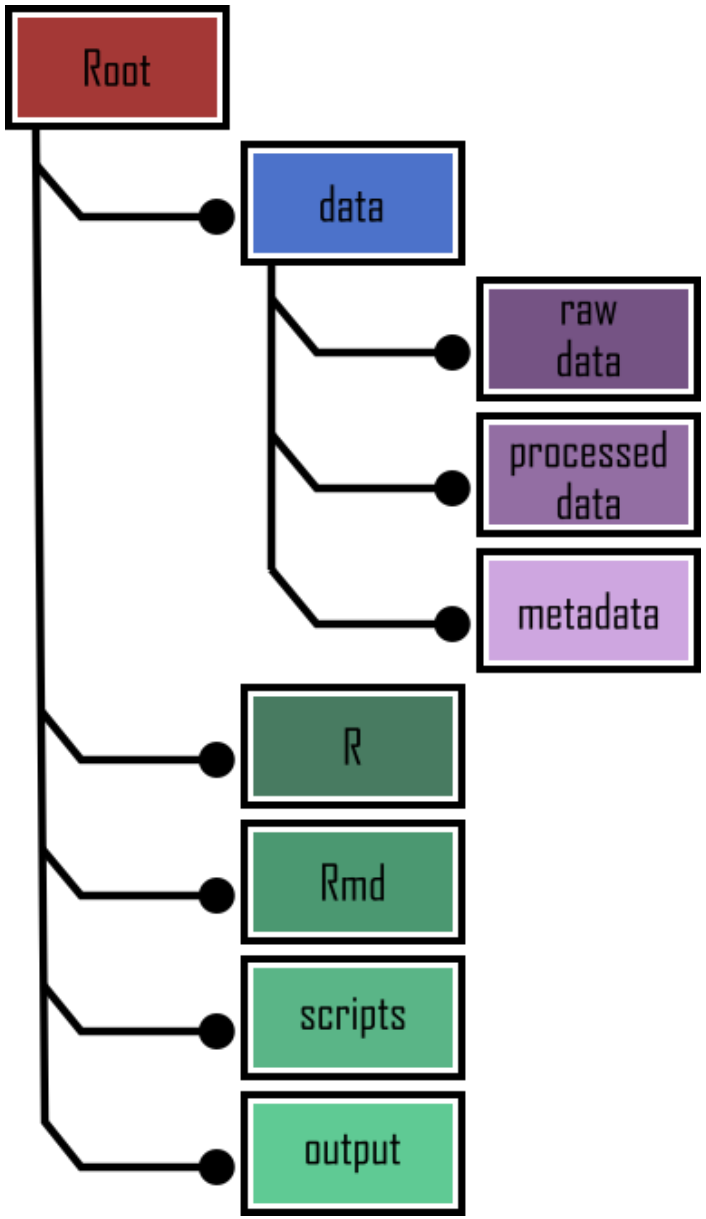
Ponovljivost: Vi (čez npr. dve leti) ali kdorkoli drug lahko celotno analizo ponovi, korak po korak, in pri vsakem koraku razume kaj se dogaja.

Razumljivost: Celoten projekt, skupaj s kodo, je logično strukturiran in dokumentiran.

Vsak projekt v svoji mapi!!
(potem pa struktura map)

- R koda
- Podatki
- Rezultati
- …

- Uporabljajte **R Markdown**, da si beležite celoten postopek.
- **Pišite** kaj počnete!!!!



Organizacija dela - povzetek

- Imejte vse v **eni mapi**. Podatke, R kodo, Markdown dokumente, rezultate (tabele, grafe, poročila…)
    - Pri kompleksnih projektih si znotraj projektne mape uredite naprej po mapah.
- Uporabljajte **R Markdown**, da si beležite celoten postopek.
- **Pišite** kaj počnete!!!!

Čitljivost R kode in zakaj bi se ukvarjali s tem?

- Jasna logika – enostavnejše programiranje
- Lažje iskanje napak (debugging)
- Prenosljivost kode
- Ponovna uporaba

```{r}
#' calculates metric distance between two points

getDistance = function(X1, X2, Y1, Y2) {
  return(sqrt((X1 - X2)^2 + (Y1 - Y2)^2) / 1000)
}
```

```{r}
sampledata$sdist = rep(NA, nrow(sampledata))
sampledata$tdist = rep(NA, nrow(sampledata))

for (i in 2:nrow(sampledata)) {
  if (sampledata$AnimalRef[i] == sampledata$AnimalRef[i - 1]) {
    sampledata$sdist[i] = getDistance(sampledata$X[i],
                                      sampledata$X[i - 1],
                                      sampledata$Y[i],
                                      sampledata$Y[i - 1])

    sampledata$tdist[i] = sampledata$Date[i] - sampledata$Date[i - 1]
  }
}


```

Lepo pisanje programske kode je podobno
uporabi ločil v običajnih besedilih...

... lahko shajamo brez...

...ampakdefinitivnopostanevseskupajmnogomančitljivo

```{r}
#' calculates metric distance between two points

getDistance = function(X1, X2, Y1, Y2) {
  return(sqrt((X1 - X2)^2 + (Y1 - Y2)^2) / 1000)
}
```

```{r}
sampledata$sdist = rep(NA, nrow(sampledata))
sampledata$tdist = rep(NA, nrow(sampledata))

for (i in 2:nrow(sampledata)) {
  if (sampledata$AnimalRef[i] == sampledata$AnimalRef[i - 1]) {
    sampledata$sdist[i] = getDistance(sampledata$X[i],
                                      sampledata$X[i - 1],
                                      sampledata$Y[i],
                                      sampledata$Y[i - 1])

    sampledata$tdist[i] = sampledata$Date[i] - sampledata$Date[i - 1]
  }
}


```

```{r}
#' calculates metric distance between two points

getDistance = function(X1, X2, Y1, Y2) {
  return(sqrt((X1 - X2)^2 + (Y1 - Y2)^2) / 1000)
}
```

```{r}
sampledata$sdist = rep(NA, nrow(sampledata))
sampledata$tdist = rep(NA, nrow(sampledata))

for (i in 2:nrow(sampledata)) {
  if (sampledata$AnimalRef[i] == sampledata$AnimalRef[i - 1]) {
    sampledata$sdist[i] = getDistance(sampledata$X[i],
                                      sampledata$X[i - 1],
                                      sampledata$Y[i],
                                      sampledata$Y[i - 1])

    sampledata$tdist[i] = sampledata$Date[i] - sampledata$Date[i - 1]
  }
}
```

```{r}
getDistance = function(X1, X2, Y1, Y2) {return(sqrt((X1 - X2)^2 + (Y1 - Y2)^2) / 1000)}

sampledata$sdist = rep(NA, nrow(sampledata))
sampledata$tdist = rep(NA, nrow(sampledata))

for (i in 2:nrow(sampledata)) {if(sampledata$AnimalRef[i] == sampledata$AnimalRef[i-1]){
    sampledata$sdist[i] =
getDistance(sampledata$X[i],sampledata$X[i-1],sampledata$Y[i],sampledata$Y[i-1])
    sampledata$tdist[i] = sampledata$Date[i] - sampledata$Date[i - 1]}}
```

```{r}
#' calculates metric distance between two points

getDistance = function(X1, X2, Y1, Y2) {
  return(sqrt((X1 - X2)^2 + (Y1 - Y2)^2) / 1000)
}
```

```{r}
sampledata$sdist = rep(NA, nrow(sampledata))
sampledata$tdist = rep(NA, nrow(sampledata))

for (i in 2:nrow(sampledata)) {
  if (sampledata$AnimalRef[i] == sampledata$AnimalRef[i - 1]) {
    sampledata$sdist[i] = getDistance(sampledata$X[i],
                                      sampledata$X[i - 1],
                                      sampledata$Y[i],
                                      sampledata$Y[i - 1])

    sampledata$tdist[i] = sampledata$Date[i] - sampledata$Date[i - 1]
  }
}


```

Organizacija zaključenih celot v chunk-e

```{r}
#' calculates metric distance between two points

getDistance = function(X1, X2, Y1, Y2) {
  return(sqrt((X1 - X2)^2 + (Y1 - Y2)^2) / 1000)
}
```

```{r}
sampledata$sdist = rep(NA, nrow(sampledata))
sampledata$tdist = rep(NA, nrow(sampledata))

for (i in 2:nrow(sampledata)) {
  if (sampledata$AnimalRef[i] == sampledata$AnimalRef[i - 1]) {
    sampledata$sdist[i] = getDistance(sampledata$X[i],
                                      sampledata$X[i - 1],
                                      sampledata$Y[i],
                                      sampledata$Y[i - 1])

    sampledata$tdist[i] = sampledata$Date[i] - sampledata$Date[i - 1]
  }
}


```

Ločite smiselne enote s praznimi vrsticami

```{r}
#' calculates metric distance between two points

getDistance = function(X1, X2, Y1, Y2) {
  return(sqrt((X1 - X2)^2 + (Y1 - Y2)^2) / 1000)
}
```

```{r}
sampledata$sdist = rep(NA, nrow(sampledata))
sampledata$tdist = rep(NA, nrow(sampledata))

for (i in 2:nrow(sampledata)) {
  if (sampledata$AnimalRef[i] == sampledata$AnimalRef[i - 1]) {
    sampledata$sdist[i] = getDistance(sampledata$X[i],
                                      sampledata$X[i - 1],
                                      sampledata$Y[i],
                                      sampledata$Y[i - 1])

    sampledata$tdist[i] = sampledata$Date[i] - sampledata$Date[i - 1]
  }
}
```

Zamikajte vrstice!!!

Zamikanje vrstic, zaviti oklepaji in presledki

Dober primer:

```
if (condition) {
  do_something()
}
```

Zavit oklepaj na koncu kode, ki začne blok

Zavit zaklepaj v svoji vrstici, poravnan z začetkom

Slab primer:

```
if(condition){
do_something()}
```

Zamikanje vrstic, zaviti oklepaji in presledki

Dober primer:

```
if (condition) {
  do_something()
} else {
 do_something_else()
}
```

Izjema je ukaz “else”, ki se da v isto vrstico kot zaviti zaklepaj.

Slab primer:

```
if (condition) {
  do_something()
}
else {
 do_something_else()
}
```

Zamikanje vrstic, zaviti oklepaji in presledki

Dober primer:

```
if (condition) {
  do_something()
}
```

Zavit oklepaj se NE daje v svojo vrstico.

Slab primer:

```
if(condition)
{
        do_something()
}
```

Zamikanje vrstic, zaviti oklepaji in presledki

Dober primer:

```
if (condition) {
  do_something()
}
```

Koda znotraj bloka { ... } zamaknjena, običajno dva presledka.

Slab primer:

```
if(condition){
do_something()}
```

Zamikanje vrstic, zaviti oklepaji in presledki

Dober primer:

```
if (condition) {
  do_something()
}
```

Presledek za kontrolnimi ukazi (if, for ...)

Brez presledka za imenom funkcije

Slab primer:

```
if(condition){
do_something()}
```

```{r}
#' calculates metric distance between two points

getDistance = function(X1, X2, Y1, Y2) {
  return(sqrt((X1 - X2)^2 + (Y1 - Y2)^2) / 1000)
}
```

```{r}
sampledata$sdist = rep(NA, nrow(sampledata))
sampledata$tdist = rep(NA, nrow(sampledata))

for (i in 2:nrow(sampledata)) {
  if (sampledata$AnimalRef[i] == sampledata$AnimalRef[i - 1]) {
    sampledata$sdist[i] = getDistance(sampledata$X[i],
                                      sampledata$X[i - 1],
                                      sampledata$Y[i],
                                      sampledata$Y[i - 1])

    sampledata$tdist[i] = sampledata$Date[i] - sampledata$Date[i - 1]
  }
}


```

NE predolgih vrstic!!

Koda “prelomljena” na logičnih mestih (npr. za vejicami), da niso predolge vrstice (siva črta v RStudiu = 80 znakov)

Presledki okrog operatorjev (=, <-, +, -, *, itd.) in <u>ZA</u> vejicami

Dober primer:

```
average <- mean(feet / 12 + inches, na.rm = T)
```

Slab primer:

```
average<-mean(feet/12+inches,na.rm=T)
```

Imena funkcij in spremenljivk

Funkcije so **glagoli**, imena spremenljivk so **samostalniki**.

Imena funkcij in spremenljivk naj bodo <u>INFORMATIVNA!</u>

Dober primer:

```
calculateSum <- function(a, b) return(a + b)
shoe_sizes <- c(44, 38, 39, 33, 43)
```

Slab primer:

```
dummyFunction <- function(a, b) return(a + b)
my_variable <- c(44, 38, 39, 33, 43)
```

Načini za čitljivo pisanje imen funkcij / spremenljivk

alllowercase – enostavno, ampak nečitljivo. SPREMENLJIVKE

period.separated – ločeno s pikami, npr. funkcija ‘naredi.nekaj()’. FUNKCIJE

snake_case – ločeno s podčrtaji. SPREMENLJIVKE

lowerCamelCase – velike/male črke, npr ‘narediNekaj()’. FUNKCIJE

UpperCamelCase – velike/male črke, npr ‘NarediNekaj()’. FUNKCIJE

Načini za čitljivo pisanje imen funkcij / spremenljivk

alllowercase – enostavno, ampak nečitljivo. SPREMENLJIVKE

period.separated – ločeno s pikami, npr. funkcija ‘naredi.nekaj()’. FUNKCIJE

snake_case – ločeno s podčrtaji. SPREMENLJIVKE

lowerCamelCase – velike/male črke, npr ‘narediNekaj()’. FUNKCIJE

UpperCamelCase – velike/male črke, npr ‘NarediNekaj()’. FUNKCIJE

Komentarji

Komentar se naredi z znakom #, ki mu sledi presledek:

To je komentar v R kodi

Komentarji bi morali razložiti **ZAKAJ** in ne KAJ ali KAKO!!!

- Dodatne informacije. Če morajo komentarji razlagati kako dela koda, je koda slabo napisana!

- Če se uporablja **Markdown**, je bolje namen (zakaj) pisati izven chunkov kot del poročila.

- Angleščina?

While the first analyses were run assuming monogamy, the results didn't seem right. When analysis was run using a polygamy assumption, the results and their interpretation improved.

First let's check performance of the Colony run. We'll plot trace files for MCMC (which was done in three independent chains) to check for convergence.

Get intermediate data

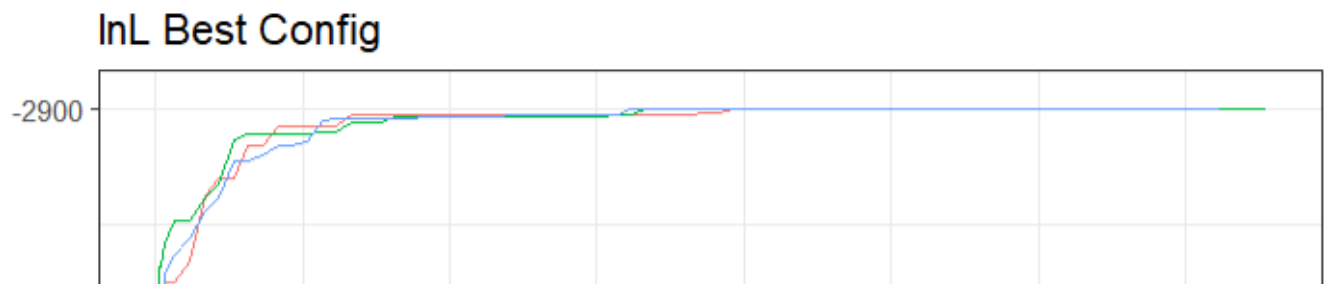
```{r}
# path to Colony results, the final version, polygamy assumed
path="./FCC2018Wolves_Poly/FCC2018Wolves_Poly"

# c/out - previous version, monogamy assumed
# path="./FCC2018Wolves_Mono/FCC2018Wolves_Mono"

midresults = read.table(paste(path,".MidResult",sep=""),header=T,sep="")
```

Graph convergence

```{r}
pm = ggplot(aes(x = NumIterate, color = as.factor(Run)), data = midresults) +
  theme_bw()
plm = pm + geom_line(aes(y = BtLogL)) + ggtitle("lnL Best Config")
plm
```



Opis (zakaj), Markdown

Komentar specifične vrstice kode.

Izključena koda s komentarjem. Dobro napisati, zakaj je koda izključena!

Komentiranje funkcij

Pri funkcijah je treba vedno razložiti kaj delajo (razen če so res trivialne).

Lahko čisto preprosto pri enostavnih funkcijah.

```{r}
#' calculates metric distance between two points

getDistance = function(X1, X2, Y1, Y2) {
  return(sqrt((X1 - X2)^2 + (Y1 - Y2)^2) / 1000)
}
```

Pri kompliciranih funkcijah temeljito komentiramo vse parametre in izhode.

Roxygen tagi

```
# ------------------------------------------------------------------------------

#' make.animal.timespan
#'
#' Produces first-seen and last-seen dates for each individual
#'
#' @param individual_id Text vector, id of individual animals
#' @param sample_date Date vector, when the sample was collected
#' @param sample_type Text vector, types of sample
#' @param dead Text vector List of sample types that are lethal
#'
#' @returns data.frame Animal id, when it was first and last seen and
#'          if it's dead


make.animal.timespan = function(individual_id,
                                sample_date,
                                sample_type,
                                dead = "Tissue") {

  unique_ind = unique(individual_id)
  out_individual = NULL
  out_firstseen = NULL
  out_lastseen = NULL
  out_isdead = NULL

  for (individual in unique_ind) {
    ind_dates = sample_date[individual_id == individual]
    ind_sample_types = sample_type[individual_id == individual]

    out_individual = c(out_individual, individual)
```

Funkcije naj uporabljajo samo **lokalne spremenljivke**, ne globalnih!

Dobro

```
my_sum <- function(a, b){
  return(a + b)
}
```

Slabo

```
a <-  10

my_sum <- function(b){
  return(a + b)
}
```

Takšna funkcija ni prenosljiva in lahko povzroča velike probleme!

Potrudite se pisati čitljivo kodo...

...vsaka minuta, ki jo boste porabili za to, se vam bo večkratno obrestovala!